

University of Dhaka

Department of Computer Science and Engineering

Lab 4 : Software Design Document (SDD)

Digital Knowledge Platform

Team: DU_Ironclad

Team Member	Class Roll	Email
Ashraful Alam	02	nafim1703@gmail.com
Md. Shahriar Kabir	20	mdshahriar- 2021711196@cs.du.ac.bd
Faiaz Mahmud	46	faiaz-2021711222@cs.du.ac.bd
Tamim Dewan Zihad	53	tamim-2021011229@cs.du.ac.bd

1 Team and Project Information

Field	Value
Team	DU_Ironclad
Project Name	Digital Knowledge Platform
Repository URL	https://github.com/shahriarkabir280/Digital-Knowledge-Platform
Date	April 9, 2026
Related SRS version	2.0

2 System Overview

Problem Summary

- Existing institutional knowledge systems are fragmented across archive, repository, library, and project showcase modules.
- Fragmentation reduces discoverability, research visibility, and operational efficiency for students, faculty, and staff.
- Current workflows lack integrated collaboration, bilingual intelligence, analytics, and preservation-aware management.

Target Users

- **Students (Member role):** Access learning resources, research papers, project showcases, and collaboration tools.
- **Researchers/Faculty (Contributor role):** Submit outputs, discover related work, and analyze citation and impact trends.
- **Librarians (Staff role):** Manage catalog and circulation workflows with reporting, imports, and barcode/QR operations.
- **Archive Managers (Staff role):** Preserve institutional documents with meta-data, integrity checks, and migration planning.
- **Administrators (Administrator role):** Govern users, permissions, analytics, compliance, and overall system operations.
- **General Public (Guest role):** Search and view publicly published institutional knowledge resources.

Project Goals

- **Archive Upload & Metadata Editor:** Develop a robust ingestion system for multi-format media with a rich metadata editor, supporting bulk uploads and automated field mapping for efficient archival.
- **Public Search/Browse & Media Viewer:** Implement an intuitive, bilingual (Bangla-English) discovery interface with an integrated high-performance viewer for PDFs, images, and streaming audio/video content.
- **Research & Showcase Pages:** Create specialized, high-visibility portals for institutional research outputs, laboratory portfolios, and academic project showcases to highlight innovation and excellence.
- **Public Catalog Portal:** Deliver a centralized, user-friendly digital library catalog with advanced filtering, allowing users to browse physical and digital collections in real-time.
- **Member Borrowing Workflow:** Automate the complete circulation lifecycle, including digital hold requests, wishlist management, personal history tracking, and automated due-date notifications.
- **Librarian Operations Dashboard & Reports:** Provide a comprehensive administrative command center for CRUD operations, inventory management, and the generation of detailed analytics and usage reports.

Out of Scope

- Course/LMS delivery and grade management are excluded from this platform scope.
- Native mobile apps are excluded for Phase 1, with responsive web as the delivery mode.
- Building a video conferencing platform or full communication suite is outside scope.
- Payment gateway integration and hardware procurement are excluded from the software MVP.

3 Architecture Design

3.1 High-Level Architecture

Main components/services:

- **Auth Service:** Handles login, password reset, session/token lifecycle, and role lookup.
- **Repository Service:** Manages research/project submissions, metadata, versions, and review state.
- **Library Service:** Manages catalog CRUD, circulation workflows, fines, and hold requests.
- **OCR/AI Worker:** Runs asynchronous OCR, summarization, recommendation, and plagiarism jobs.
- **Search Service:** Provides bilingual indexed search and filtering for all discoverable content.
- **Notification Service:** Sends event-driven emails for review updates, due reminders, and alerts.
- **Analytics Service:** Aggregates usage, research trends, and operational KPIs for dashboards.
- **Data Layer:** PostgreSQL (transactional), Redis (cache), Queue broker, and object storage.

Data Flow Summary:

- **User Requests** from the Web UI pass through an **API Gateway** to be **authenticated** and routed to the correct **domain services**.
- **Domain services** then **save transactional data** into the **database** and **store files** in **object storage**.
- Heavy **AI or OCR tasks** are sent to a **queue** to be processed **asynchronously** by separate **worker nodes**.
- Once complete, the **workers update search indexes** and **trigger notifications** or **analytics events**.
- For **fast performance**, **search requests** and **dashboards** read directly from these **optimized data stores**.

External integrations (if enabled):

- Optional Phase 2 cloud APIs: OpenAI, Google Cloud Vision/NLP, Turnitin/Copyleaks.
- Optional institutional integrations: SSO/IdP, legacy library DB import, SMTP relay.

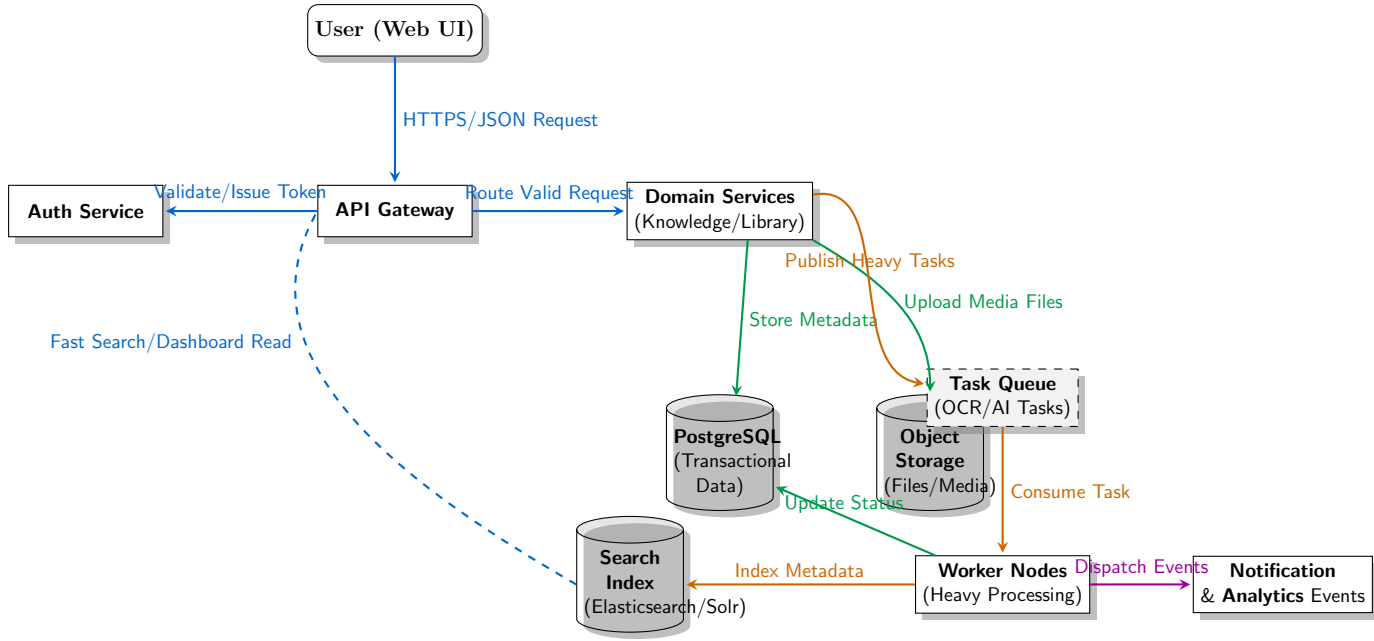


Figure 1: Detailed data flow for request handling, persistence, asynchronous processing, and user-facing read models.

3.2 Component Breakdown

Component	Responsibility	Inputs	Outputs	Notes
Auth Service	Authentication, session control, role resolution	Email/password, reset tokens, session refresh requests	JWT/session token, user role claims, auth events	Enforces inactivity timeout and warning policy
Repository Service	Submission workflow, metadata/version control, review routing	Paper/project files, metadata forms, review actions	Submission records, version history, review states	Supports draft/review/published lifecycle
Library Service	Catalog and circulation operations	Catalog records, barcode/QR scans, checkout/return commands	Loan records, due dates, fine calculations, hold queues	Integrates with overdue notification triggers
OCR/AI Worker	Async OCR and AI processing	Queue jobs, document binary, model config	OCR text, summaries, similarity scores, recommendation vectors	Queue-based processing with SLA monitoring

Component	Responsibility	Inputs	Outputs	Notes
Search Service	Bilingual indexing and query execution	Normalized metadata, OCR text, user search query/filter params	Ranked results, facets, suggestion terms	Bangla/English indexing strategy
Notification Service	Event-driven messaging	Domain events (due, hold, review, completion), templates	Email notifications, delivery logs	Retries and dead-letter handling
Analytics Service	Metrics aggregation and reporting	Usage events, citation stats, circulation data	Dashboard KPIs, trends, leaderboard datasets	Aggregated/anonymized where required
PostgreSQL DB	Source of truth for transactional domain data	CRUD operations from domain services	Persisted records, relational joins, audit rows	PK/FK constraints and migration-controlled schema
Redis Cache	Fast lookup/session/cache layer	Hot query keys, session entries, computed fragments	Cached responses, reduced DB read load	TTL-based invalidation strategy
Queue Broker	Decouples synchronous API from async workers	Job publish requests, retry policies, priorities	Worker-ready jobs, retry/dead-letter queues	Required for OCR/AI/background jobs
Object Storage	Durable storage for uploaded files and exports	File uploads, transformed derivatives, backup artifacts	File URIs, signed download links, versioned blobs	Supports large files and retention rules

3.3 Key Design Decisions

Decision	Options Considered	Chosen Option	Reason
Deployment Model	Cloud-first, On-prem only, Hybrid phased	On-prem MVP + optional cloud Phase 2	Satisfies institutional constraints now while keeping future integration flexibility.
AI/OCR Execution Mode	Inline synchronous, Async background jobs, External-only service	Async background jobs via worker queue	Prevents UI/API latency spikes and supports retry/monitoring for long-running tasks.

Decision	Options Considered	Chosen Option	Reason
Authorization Strategy	Coarse role checks, Attribute-based access, Role-based access control (RBAC)	RBAC with scoped permissions per module	Matches defined user roles and provides predictable access governance.
Queue Architecture	No queue, In-memory job runner, Dedicated queue broker + workers	Dedicated queue broker + worker services	Decouples API from heavy jobs and enables scalability, backoff, and dead-letter handling.
Data Storage Pattern	Single relational DB only, Polyglot persistence, File-system only	Polyglot persistence (PostgreSQL + Redis + object storage)	Balances consistency, performance, and large-file handling requirements.
Search Architecture	SQL full-text only, External index service, Manual metadata filtering	External indexed search service (bilingual)	Improves relevance, filtering, and response time for Bangla-English discovery.

4 Data Design

Core Entities

Entity	Purpose	Primary Keys / FKs	Implementation Notes
User	Platform identity and role ownership	user_id (PK)	Email unique, role enum (guest/member/contributor/staff/admin/rev) status tracked.
Document	Canonical digital content object	doc_id (PK), uploader_id (FK User)	Stores title/type/format/path/version/state and access_tier for authorization checks.
Metadata	Rich descriptive fields for document discovery	metadata_id (PK), doc_id (FK Document, unique)	One-to-one with Document; supports JSON and structured fields for search indexing.
Catalog Item	Library inventory item (physical or digital)	item_id (PK)	Barcode unique; status enum controls circulation and availability.
Loan	Borrowing transaction record	loan_id (PK), item_id (FK), member_id (FK User)	due_date mandatory for active loans; supports renewals, return state, fine calculation.

Entity	Purpose	Primary Keys / FKs	Implementation Notes
Annotation	User comments/highlights on documents	annotation_id (PK), doc_id (FK), user_id (FK), parent_id (self FK)	Location stored as page + coordinates; supports threaded replies and visibility scopes.
Research Paper	Scholarly subtype of document	paper_id (PK), doc_id (FK), lab_id (FK)	Citation count, author list, and submission/review attributes for repository workflows.
Lab	Research group/department profile	lab_id (PK), head_id (FK User)	Head must map to faculty/contributor authority; linked to publications and analytics.
Citation	Citation network edge between papers	citation_id (PK), source_paper_id (FK), target_paper_id (FK)	Supports many-to-many self-referencing citation graph queries.
Preservation	File integrity and migration tracking	preservation_id (PK), doc_id (FK)	Stores checksum history, risk flags, migration recommendation/status, and actions log.

Key Relationships

- One User uploads many Documents; each Document belongs to exactly one uploader.
- One Document has exactly one Metadata record (one-to-one cardinality).
- One Document has many Annotations; one User can create many Annotations.
- One CatalogItem has many Loan records over time; active loan count is constrained by availability.
- ResearchPaper has many-to-many self-citations through Citation edges.
- One Lab has many ResearchPapers; each ResearchPaper belongs to one Lab.
- One Document can have many Preservation records for periodic integrity and migration tracking.

Constraints and Validation Rules

- User email must be unique and conform to institutional email format policy.
- Document file_size must be $\leq 500\text{MB}$ and format must be in the allowed media list.

- Document state must be one of: `draft`, `review`, `published`, `archived`.
- `access_tier` must be one of: `public`, `member`, `staff`, `restricted`.
- Barcode values in `CatalogItem` must be unique across the catalog.
- For Loan, `due_date` must be greater than `checkout_date` and required while loan is active.
- Annotation visibility must follow role/access policy before rendering to other users.
- Audit-sensitive operations must write immutable log entries with timestamp and actor reference.

Data Lifecycle and Retention/Deletion Rules

- Document lifecycle: `draft` → `review` → `published` → `archived`.
- Each document update creates a new version; previous versions remain restorable for authorized staff.
- OCR/AI outputs are persisted as derived metadata and can be recalculated when models are updated.
- Backup lifecycle follows daily incremental and weekly full snapshot policy.
- Activity/audit logs are retained for 1 year to satisfy operational traceability requirements.
- GDPR-style deletion requests remove personal profile/content ownership data where applicable.
- When deletion is requested, audit logs are retained but user identifiers are anonymized (hashed).
- Archived and preservation-marked records remain queryable to staff per access tier and policy.

5 Core Flow Design

Flow 1: Research Submission + Plagiarism + Summary + Review

Trigger: A Contributor/Researcher starts a new paper submission from the repository module.

Steps:

1. User opens **Research Repository > Submit Paper** and uploads PDF/DOCX.
2. System validates file type/size, performs malware scan, and stores the file in object storage.
3. System extracts metadata (title, authors, abstract, keywords) and pre-fills submission form.
4. User reviews/edits metadata, selects lab/department, access tier, and submits for processing.
5. System enqueues plagiarism and AI summarization jobs in queue broker.
6. Worker executes plagiarism check and generates summary/key findings asynchronously.
7. System updates submission record with score, summary, and processing timestamps.
8. User reviews generated outputs and either accepts or edits summary before final submit.
9. System assigns submission ID, sets state to **review**, and notifies assigned reviewers.

Success outcome:

- Submission is stored with searchable metadata, review state, and versioned artifacts.
- Plagiarism report and summary are attached to the submission record.
- Reviewer notification is sent and audit log entry is created.

Failure/edge cases:

- Invalid file format or size limit exceeded → reject upload with actionable validation message.
- Plagiarism service timeout → keep submission in pending-processing state and retry from queue.
- Plagiarism score above policy threshold → show warning, require user confirmation/manual review path.

- Summary generation fails → allow manual summary entry; record failure event in logs.
- Reviewer assignment unavailable → route to staff review pool and notify administrator.

Flow 2: Library Checkout/Return + Due Tracking/Notification

Trigger: Librarian scans member ID and item barcode/QR at circulation desk.

Steps:

1. Librarian authenticates and opens circulation interface.
2. System reads member profile, role/status, loan limits, fines, and active holds.
3. Librarian scans item code; system fetches item availability and policy constraints.
4. On checkout, system creates **Loan** record, computes due date, updates item status to **checked_out**.
5. Notification service sends checkout confirmation with due date and renewal rules.
6. Scheduler evaluates due-tracking jobs daily and triggers reminder/overdue notifications.
7. On return scan, system closes loan, computes overdue fine (if any), and sets item to **available**.
8. System records circulation/audit events and refreshes member history/reporting aggregates.

Success outcome:

- Loan lifecycle is accurately recorded from checkout to return.
- Member receives due/overdue notifications at configured schedule.
- Item status, fines, and borrowing history remain consistent across modules.

Failure/edge cases:

- Member exceeds loan limit or has blocking fines → block checkout and show override policy.
- Item already checked out → prevent checkout and offer hold placement workflow.
- Notification delivery failure → retry with backoff; keep delivery log and alert staff if exhausted.
- Return scan mismatch or unreadable code → fallback to manual lookup with dual confirmation.
- Due-tracking job delay → run catch-up processing and mark delayed notifications in ops log.

Flow 3: Fine Calculation + Payment/Settlement Recording

Trigger: Overdue return or damaged item event is detected during circulation processing.

Steps:

1. Librarian opens the active loan record from circulation history.
2. System computes overdue duration from `due_date` and return timestamp, then applies configured fine rules.
3. If damage is reported, staff selects damage category and system adds policy-based damage charge.
4. System generates a provisional settlement summary (overdue fine + damage charge + waivers/discounts if authorized).
5. Librarian confirms settlement and records payment method (cash, card, or institutional wallet/manual ledger).
6. System marks settlement status as `paid`, `partially_paid`, or `waived` with approver metadata.
7. Member account balance, borrowing eligibility flags, and audit logs are updated.
8. Receipt/confirmation is issued and optional notification is sent to the member.

Success outcome:

- Fine and settlement records are stored with traceable calculation inputs and approval metadata.
- Member liability state and circulation eligibility are consistent across account and loan modules.
- Audit trail captures settlement actor, timestamp, amount breakdown, and status transition.

Failure/edge cases:

- Loan record missing/closed inconsistency → block settlement, require staff reconciliation path.
- Policy configuration unavailable → fallback to default fine rule and flag for admin review.
- Partial payment recorded → keep residual balance, mark account as restricted by policy threshold.
- Unauthorized waiver attempt → reject action and require elevated approver role.
- Receipt generation or notification failure → retain settlement commit and enqueue retry event.

Flow 4: Advanced Search + Faceted Filtering + Access-Tier Enforcement

Trigger: User submits a multilingual search query with optional filters/facets.

Steps:

1. User enters Bangla/English keyword query and selects optional filters (type, year, lab, author, access tier).
2. Search service normalizes text (tokenization/stemming/synonym expansion) and builds ranked query plan.
3. System resolves user role/session claims and derives allowed visibility scope from RBAC and access tiers.
4. Search engine executes full-text and facet aggregation query across indexed meta-data/OCR fields.
5. Result set is post-filtered to enforce permission rules before rendering snippets and file actions.
6. System returns ranked results with facet counts, highlighted matches, and sort options.
7. User applies additional filters or opens a result; access-controlled content view is served accordingly.
8. Query and click events are logged for analytics and relevance tuning.

Success outcome:

- User receives relevant multilingual results with fast response and accurate facet counts.
- Restricted content is never exposed beyond role and access-tier policy.
- Search telemetry is captured for dashboard metrics and ranking improvements.

Failure/edge cases:

- Empty or invalid query → return validation prompt and suggestion terms.
- Search index lag/staleness → show last-indexed timestamp and allow retry/refresh path.
- Unauthorized direct resource open → deny with access message and log security event.
- High-latency query under heavy load → return partial results with degraded-mode notice.
- Mixed-language tokenization miss → provide fallback fuzzy match and related-term suggestions.

6 Security and Access Design

Authentication Approach

- Email/password authentication with institutional account policy and optional MFA for staff/admin roles.
- Session/JWT tokens with secure storage, inactivity timeout at 30 minutes, and warning at 2 minutes before expiry.
- Password reset uses single-use, time-bound tokens and audited recovery events.

Authorization Model (Roles/Permissions)

- RBAC is enforced at both API and data access layers for Guest, Member, Contributor, Staff, Admin, Reviewer.
- Access tiers (public/member/staff/restricted) are checked before file, metadata, and annotation visibility.
- High-risk actions (state transitions, deletion, bulk import, policy updates) require elevated role checks.

Sensitive Data Handling

- TLS 1.3 for data in transit and encrypted storage for sensitive records at rest.
- Uploaded files validated by MIME/type/size checks and malware scanning before persistence.
- PII minimization applied in analytics outputs; only aggregated/anonymized data exposed in dashboards.
- User deletion requests remove personal profile-linked data while preserving policy-required records.

Audit/Logging Plan (Contradiction-Safe)

- Log all sensitive operations with timestamp, actor role, action, and target object reference.
- Retain audit logs for 1 year for traceability, incident response, and compliance review.
- On deletion requests, direct identifiers in logs are anonymized (hashed) rather than fully erased.
- Maintain immutable append-only log store with access limited to authorized security/staff roles.

7 Implementation Plan

Sprint 1 Completed Features

- Project foundation: architecture baseline, service boundaries, and environment setup finalized.
- Authentication and RBAC baseline implemented for Guest, Member, Contributor, Staff, Admin, Reviewer roles.
- Repository submission pipeline baseline implemented (file upload, metadata capture, draft/review state handling).
- Library circulation baseline implemented (catalog operations, checkout/return records, due-date core logic).
- Asynchronous job pipeline skeleton implemented (queue broker + worker pattern for OCR/AI tasks).
- Search indexing baseline established for core metadata and document discoverability.

Pending Features for Sprint 2

- Full OCR and summarization integration with benchmark-driven quality validation pipeline.
- Plagiarism processing workflow hardening with policy thresholds, retries, and reviewer escalation.
- Notification service completion for due reminders, review notifications, and delivery retry handling.
- Analytics dashboards for circulation, research trends, and contribution metrics.
- Preservation workflows (checksum monitoring, migration recommendations, and archival action logging).
- Compliance hardening (deletion workflow automation, log anonymization checks, and audit report export).

Technical Risks and Mitigation Plan

Risk	Probability Impact		Mitigation
OCR quality below target CER	Medium	High	Benchmark multiple OCR configs (BanglaOCR dataset), maintain correction UI, and tune preprocessing pipeline.
Queue backlog under peak processing load	Medium	High	Enforce worker autoscaling policy, priority queues, retry with backoff, and dead-letter monitoring.
On-prem resource bottlenecks (CPU/storage)	Medium	High	Capacity planning before Sprint 2, staged load tests, and storage quota/retention controls.
Notification failures reduce user trust	Medium	Medium	Add retry policy, fallback channels, delivery audits, and ops alerting for repeated failures.
Compliance conflict in deletion vs logs	Low	High	Apply strict anonymization for retained logs, automate deletion workflow, and add compliance test cases.
Integration delays for optional cloud APIs	Low	Medium	Keep self-hosted MVP path fully functional and gate cloud connectors to Phase 2 only.

8 Testing Plan (Design Level)

Unit Testing Targets

- **Auth Service:** credential validation, token issue/refresh/revoke, role claim parsing, timeout warning logic.
- **Repository Service:** metadata validation, lifecycle state transitions, version increment logic, access-tier checks.
- **Library Service:** checkout/return rules, due-date calculator, renewal limits, fine computation, hold queue behavior.
- **OCR/AI Worker:** job payload validation, retry/backoff decisions, result parser, fallback to manual summary path.
- **Search Service:** query normalization, bilingual tokenization helpers, ranking filter composition.
- **Notification Service:** template rendering, recipient policy checks, retry scheduling, dead-letter routing rules.

- **Analytics Service:** event aggregation functions, metric calculators, anonymization utilities.
- **Data Layer Adapters:** repository methods, cache key TTL behavior, object-storage URI signing helpers.

Integration Testing Targets

- Auth + RBAC + API gateway: protected endpoint access by role and tier-based content visibility.
- Repository + Queue + Worker: submission event creates jobs; worker updates status/artifacts in repository.
- Library + Notification: checkout/return updates trigger due reminders and overdue notification schedule.
- Repository + Search: metadata/OCR outputs are indexed and become queryable with expected filters.
- Worker + Storage + DB: OCR/summary artifacts persisted and linked to correct document versions.
- Compliance integration: deletion request pipeline anonymizes audit log identifiers while preserving 1-year retention.
- Observability integration: processing failures produce logs, metrics, and alert events with correlation IDs.

E2E Testing Targets

- **E2E-01 Submission Journey:** contributor uploads paper, receives plagiarism/summary outputs, submits to review.
- **E2E-02 Review Journey:** reviewer receives assignment, posts feedback, and changes recommendation state.
- **E2E-03 Circulation Journey:** librarian checkout/return flow including due tracking and reminder notifications.
- **E2E-04 Discovery Journey:** user performs bilingual search and opens accessible content by role/tier.
- **E2E-05 Compliance Journey:** user deletion request processed with retained but anonymized audit traces.
- **E2E-06 Session Journey:** inactivity timeout and 2-minute warning behavior validated end-to-end.

Known Hard-to-Test Areas (AI/OCR and Async Queue)

- Non-deterministic AI outputs can cause flaky assertions; use threshold-based checks and golden reference sets.
- OCR quality varies by scan quality/font; validate with benchmark datasets and CER distribution, not single samples.
- Queue timing/race conditions (duplicate jobs, out-of-order completion) require deterministic test harness and idempotency checks.
- Retry/dead-letter behavior is time-dependent; use controllable clocks or shortened test retry windows.
- External optional APIs (Phase 2) may be unavailable in CI; test via mocks/contracts and run limited staging smoke tests.

Test Environment and Exit Signals

- Unit tests run on every merge request; integration suite runs on main branch and release candidates.
- E2E suite runs against staging with production-like queue/storage configuration.
- Exit criteria: critical E2E paths pass, no unresolved high-severity defects, and benchmark thresholds met for OCR/summary quality.